DASHBOARD OPERATING GUIDE

NYC311 Hourly Workforce Optimization

Hourly demand proxy, shift assignments, budget constraints, and service shortfalls

Prepared 2026-10-24 | Detailed user, data, model, and operational reference

Local URL	http://localhost:8051/
Launch	.venv/bin/python src/workforce_app.py
Source	Live NYC Open Data aggregates by request date, hour, and borough
Optimizer	Two-stage SciPy HiGHS mixed-integer linear program

Important data and decision boundary

NYC 311 service-request records are not equivalent to phone-call arrivals, agent schedules, or payroll. Forecast conversions and labor inputs are scenarios unless calibrated against authorized operational records. Dashboard output is planning support and requires human review.

1. Purpose and safe interpretation

This standalone Dash application creates a next-day hourly staffing scenario. It forecasts hourly NYC 311 service-request creation, converts that request proxy to a configurable call-demand scenario, allocates agents to eligible shifts, and reports expected coverage and spend.

The source dataset does not contain ACD call arrivals, agent rosters, average handle-time records, skills, breaks, wages, or labor contracts. Forecast calls, available headcount, hourly wage, shift eligibility, and capacity assumptions are therefore scenario inputs. Do not treat an output as an approved production schedule.

- The default calls-per-request factor is 0.05; replace it with a factor validated against call-center telemetry.
- Call allocations are same-hour planning estimates, not a queue or backlog simulation.
- A roster upload can describe eligible shifts and agent-specific limits, but not all workforce policies.

2. Data loading and hourly forecast

On launch, the app queries up to 120 recent days from the NYC Open Data 311 endpoint, grouped by day, hour, and borough. The page reports the most recent source date. Refresh live request data to replace the in-browser hourly data store with a fresh aggregate.

For each target hour, the forecast uses the average of recent observations at the same hour and weekday, with an eight-week lookback. The rolling-origin validation evaluates recent held-out dates and reports MAE and RMSE after scaling to the chosen calls-per-request scenario.

This is an interpretable seasonal baseline, not AutoARIMA, Holt-Winters, LSTM, or an uncertainty-calibrated demand model. Borough selection filters the history before forecasting. The selected target date must be later than the latest observed date for that borough.

3. Scenario controls

Select a borough and target forecast date. Set calls per service request, available agents, eligible shift starts, shift duration, daily budget, hourly wage, average handle time, safe occupancy ceiling, maximum per-shift workload, and optional minimum per-shift workload. An optional CSV roster replaces the interchangeable-agent pool.

The roster requires agent_id, eligible_shifts, hourly_wage, and max_calls_per_shift; min_calls_per_shift is optional. eligible_shifts is a comma-separated list of start hours. Each agent can receive at most one eligible shift. Malformed files and invalid bounds produce explicit validation errors.

The model uses hourly capacity equal to 60 divided by average handle time, multiplied by the occupancy ceiling. A half-capacity midpoint hour approximates a break. Without a roster, all agents share the same wage, shift eligibility, and workload limits.

- Minimum per-shift calls is zero by default. A positive value may be infeasible when demand is low and should only be enabled when justified.
- Shifts must fit within the modeled 24-hour day; the standard starts are 00:00, 04:00, 08:00, 12:00, and 16:00.
- An hourly per-agent capacity is needed to protect service and workload; a per-shift cap alone is insufficient.

4. Optimization logic and outputs

The SciPy HiGHS MILP has binary agent/shift choices, integer calls assigned to each agent/hour, and integer unserved demand. It enforces exact hourly demand balance, hourly on-duty capacity, per-shift maximum and optional minimum calls, one shift per agent, and a hard total labor budget.

Optimization is lexicographic: first minimize total same-hour unserved calls; then, without allowing worse service, minimize the cost of complete shifts. A budget-constrained shortfall is reported rather than hidden by overloading agents. Shift costs equal hourly wage multiplied by the complete shift length.

The chart shows forecast calls, calls assigned, uncovered calls, and scheduled agents. KPI cards show selected shifts, spend, remaining budget, solver status, and unmet demand. The schedule table presents agent/shift assignments and the hourly detail table reports each hour's capacity and service shortfall.

5. Running a scenario and diagnosing issues

Press Forecast and optimize after reviewing all controls. The output includes rolling-origin error metrics and the configured request-to-call factor. A selected roster is used for the current browser session only; the app does not publish or persist an approved roster.

If no shifts are affordable, lower demand assumptions only when supported by evidence, increase the authorized budget, expand availability, or arrange overflow handling. Never relax safe capacity or contractual limits solely to suppress an alert.

If live data fails, verify network access to data.cityofnewyork.us and retry. If no borough data exists, choose another borough or ALL. If the solver cannot find a feasible result, inspect its error and confirm budget, roster, shift starts, minimum/maximum workload, and agent availability.

6. Important production gaps

Before deployment, incorporate actual ACD arrival data, intraday seasonality and forecast error buffers, handling-time distributions, skills, breaks and rest rules, cross-midnight shifts, agent availability, contracts, schedule lock-in, queue/backlog carryover, abandonment, and service-level requirements.

The current model uses a single-day horizon and treats unserved calls as a same-hour service shortfall. It does not model queue carryover, dynamically re-optimize live arrivals, authenticate users, audit approvals, or write a published schedule.